Sustain Sync

Course: CS 4850 - 02
Semester: Fall 2025
Project-ID: Indy-2
Authors: Youssef El-Shaer & Zaid Khan
Professor: Sharon Perry
12/8/2025

Stats & Status	
LOC	8732
Components & Tools	8 : Canva, React Native, Docker, Django, RAG, Ollama, Github Actions, PostgreSQL
Hours Estimate	287
Hours Actual	289.76
Status	Project is 90% complete and is working as expected

Website: https://sustainsync.github.io/SustainSync-Website/
Github: https://github.com/SustainSync/cs4850/tree/main/SustainSync

Table of Contents

Abstract

This project focuses on the broader eco-sustainability landscape within businesses. The initiative began with the observation that no universal standard or metric exists to evaluate corporate sustainability performance. As a starting point, the system analyzes a resource shared by all organizations—utility bills. By leveraging general usage data found in these documents, the MVP establishes a foundation for a comprehensive RAG pipeline, forecasting metrics, and AI-driven sustainability recommendations.

Sustain Sync applies machine learning to analyze and forecast environmental impacts by ingesting structured and semi-structured data from utility bills in CSV formats. This input is normalized into a consistent schema for further processing. The MVP includes two major components: a forecasting module that predicts trends in CO_2 emissions, water consumption, and energy usage, and a correlation module that uses large language models to identify relationships between sustainability actions across domains such as carbon, biodiversity, and water. Contextual variables, including location and energy sources, are incorporated to enhance accuracy.

A user-friendly web interface enables data uploads and access to historical dashboards, while an internal API provides structured outputs for integration with our dashboards and reporting pipelines. MVP success criteria include the generation of forecasted trends and the identification of meaningful cross-domain correlations, establishing a scalable framework for data-driven sustainability insights.

Challenges and Assumptions

1.1 Challenges

- **Hardware Constraints:** Our development machine's limited GPU (8GB RTX 4060 Ti) created performance bottlenecks for training forecasting models and generating RAG embeddings.
- **Small Team Size:** With only two developers, we had to aggressively prioritize tasks to deliver an end-to-end platform within the semester timeline.
- **Shift to CSV-Only Ingestion:** Due to time constraints and the complexity of PDF parsing, we pivoted to supporting CSV ingestion only, which required redesigning several parts of our initial data ingestion plan.
- **Strict Data Requirements:** Forecasting depended on at least 12 months of valid historical data and a rigid CSV template, making validation, normalization, and error handling more complex.
- **Multi-Model Integration:** Ensuring consistency between the forecasting engine and the RAG co-benefit engine required tight coordination across backend logic, database design, and frontend visualization.

- **Architectural Complexity:** Implementing a modular system with Docker, Django REST APIs, React, and multiple ML services introduced significant engineering overhead and required careful synchronization.
- **AI Reliability:** Preventing LLM hallucinations required structured, data-anchored prompts and post-processing rules to ensure consistently grounded insights.

1.2 Assumptions

- Users will provide their own data in our **Sustain Sync CSV form template**.
- At least **12 months of historical data** is required for accurate forecasting, with 10 years being fully supported.
- Data from the CSVs will be filtered to match upload consistency, and editable after upload to ensure accuracy.

System assumes English-language input and numeric formats consistent with U.S. usage (e.g., kWh, gallons, metric tons CO_2).

Requirements (SRS)

1.0 Project Goals

- Provide a data ingestion engine that accepts utility bill data as CSV files formatted using the Sustain Sync template.
- Build a forecasting engine that applies time-series models (e.g., LSTM, Prophet) to predict future resource usage based on historical data.
- Integrate a co-benefit analysis engine (SSCE) powered by an LLM to identify correlations between sustainability initiatives across domains (CO_2, water, biodiversity).
- Offer a goal-tracking system where users can define measurable sustainability goals and track progress over time using forecasts and insights.
- Deliver results in an interactive web dashboard designed for accessibility and clarity.

2.0 Design Constraints

2.1 Environment

- Locally hosted web application with the option for cloud deployment.
- Backend: Django, scikit-learn, pytorch, pandas, Prophet, LSTM.
- Frontend: React-based interface for file upload and dashboard visualization.
- Data storage: Django databases and a postgresql database
- Version control: GitHub (Projects board for issue tracking, semantic releases for versioning).

2.2 User Characteristics

- Intended users are company sustainability teams and analysts.
- Users are expected to have basic technical literacy, including the ability to export data into the CSV template.
- The interface will abstract technical details into visual insights and actionable goals for users.

2.3 System Constraints

- System supports template CSV files only.
- All CSV inputs must conform to the Sustain Sync format (columns for date, usage values, units).
- Forecasting requires at least 12 months of valid entries.
- Outputs include visual dashboards, ai reports via the web

3.0 Functional Requirements

3.1 Data Ingestion

- **FR-1.1:** The system shall allow users to upload CSV files that follow the Sustain Sync template.
- **FR-1.2:** The system shall parse uploaded CSV files to extract billing period, usage values, units, and resource type.
- **FR-1.3:** The system shall validate uploaded CSV files against the Sustain Sync template and return clear error messages for malformed or missing fields.
- **FR-1.4:** The system shall normalize validated consumption records and store them in the PostgreSQL database.
- **FR-1.5:** The system shall prevent duplicate bill entries for the same billing period and resource type.

3.2 Historical Dashboard

- **FR-2.1:** The dashboard shall display aggregated water, gas, and electricity usage across all ingested data.
- **FR-2.2:** The dashboard shall provide resource-specific views with time-series charts of historical usage.
- **FR-2.3:** Users shall be able to view detailed monthly entries with scrolling or pagination.
- **FR-2.4**: Sustainability targets shall be displayed alongside actual usage for comparison.

3.3 Forecasting Engine

- **FR-3.1:** The forecasting engine shall generate projections for water, gas, and electricity usage using Prophet, with a linear regression fallback when needed.

- **FR-3.2:** The system shall allow users to select a forecast horizon between 1 and 12 months.
- **FR-3.3:** Forecast output shall include predicted values and confidence intervals (yhat_lower, yhat_upper).
- **FR-3.4:** Forecasts shall be displayed through visual charts and supporting tables.
- **FR-3.5:** Forecasts shall update when new data is ingested.

3.4 Co-Benefit Engine (SSCE)

- **FR-4.1:** The SSCE shall analyze correlations between sustainability data, user goals, and historical trends using a Retrieval-Augmented Generation (RAG) model.
- **FR-4.2:** The SSCE shall generate AI-driven insights and recommendations grounded in ingested data and user-defined goals.
- **FR-4.3:** The system shall provide narrative explanations that highlight key trends, trade-offs, and synergistic opportunities.

3.5 Goals Management

- **FR-5.1:** Users shall be able to define sustainability goals with numeric targets, timelines, and resource categories.
- **FR-5.2:** The system shall track progress toward goals using historical and forecasted data.
- **FR-5.3:** The SSCE shall produce recommendations aligned with the user's goals.
- **FR-5.4:** Goal progress shall be visualized using charts and summary text.

4.0 Non-Functional Requirements

4.1 Security

- **NFR-1.1:** The system should require authentication for data upload and goal management (future capability).
- **NFR-1.2:** All data transmitted between frontend and backend shall use secure channels when deployed.
- **NFR-1.3:** Sensitive environment variables (database credentials, model keys) shall be stored outside source control.

4.2 Data Quality and Integrity

- **NFR-2.1:** All ingested CSV data shall undergo schema validation before being stored.
- **NFR-2.2:** Data transformations (normalization and forecasting preparation) shall follow reproducible and deterministic logic.

4.3 Performance and Capacity

- **NFR-4.1:** The interface shall follow a clear workflow: Upload → Dashboard → Forecasts → Goals.
- **NFR-4.2:** Users shall receive immediate feedback when uploading or validating CSV files.
- **NFR-4.3:** Visual dashboards shall use accessible color schemes, legends, and labeled axes for interpretability.

4.4 Usability

- **NFR-4.1:** The interface shall follow a clear workflow: Upload → Dashboard → Forecasts → Goals.
- **NFR-4.2:** Users shall receive immediate feedback when uploading or validating CSV files.
- **NFR-4.3:** Visual dashboards shall use accessible color schemes, legends, and labeled axes for interpretability.

4.5 Reliability and Maintainability

- **NFR-5.1:** The system shall use modular services (ingestion, dashboard, forecasting, SSCE) to support future expansion.
- **NFR-5.2:** Docker-based deployment shall allow repeatable environment setup with consistent configurations.
- **NFR-5.3:** Core functionality (ingestion, forecasting, SSCE) shall include basic error handling and logging for debugging.

5.0 External Interface Requirements

5.1 User Interface Requirements

- File upload panel with drag-and-drop functionality and CSV template download link.
- Dashboard with tabs for overview, water, gas, electricity, and sustainability goals.
- Interactive graphs with options to alter the amount of data seen/used in forecasting.
- Co-benefit visualizations with clear indicators of confidence levels.

5.2 Hardware Interface Requirements

- Host system should have sufficient CPU/GPU resources for model training.
- 64gb Memory, 750gb Storage
- End users only require a **modern web browser**.

5.3 Software Interface Requirements

- CSV validation and processing (pandas, sql).
- Forecasting frameworks: Prophet, faiss, nlp.

- LLM API (Ollama) for SSCE correlation analysis.

5.4 Communication Interface Requirements

- Internal REST APIs between backend and frontend.
- Django backend for dynamic web interface

Software Architecture & Design (SDD)

1.0 System Overview

Sustain Sync ingests **standardized CSV templates**, normalizes data, and applies machine learning forecasting models alongside a co-benefit engine (SSCE) for correlation analysis. Users interact via a **web dashboard** to upload data, review historical performance, define goals, and receive actionable insights across **CO_2 emissions, water, and biodiversity.**

2.0 Design Considerations

2.1 Assumptions and Dependencies

- Users upload **CSV (Sustain Sync template)** data.
- MVP supports **single-organization use**; scalability is planned.
- External datasets (EPA, WHO, climate feeds) may be optionally integrated.
- Initial deployment will be hosted in KSU datacenter on VM.
- Python 3.12, Django, Prophet for ML/forecasting.

2.2 General Constraints

- Must support **modern browsers** for the web interface.
- Requires **64GB RAM / 750GB storage** for training environment/hosting (Handled by Sustain Sync).
- CSV inputs must follow Sustain Sync's defined template.
- Forecasting requires at least **12 months of historical data**.

2.3 Goals and Guidelines

- Accuracy: forecasts must be **statistically defensible**.
- Usability: dashboards must be interpretable by **non-technical users**.
- Extensibility: modular pipeline for adding new sustainability domains.

2.4 Development Methods

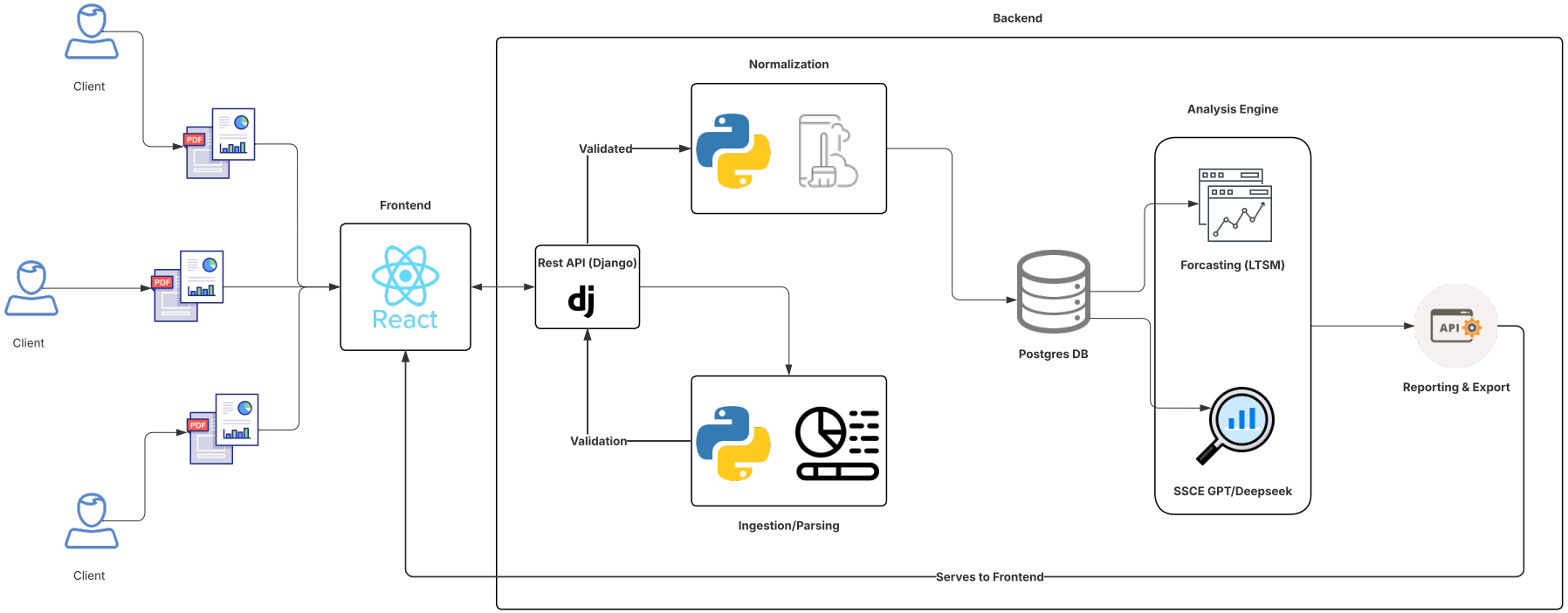
- Agile methodology with **weekly sprints**.

- GitHub for **version control** (vX.0.0 for major, vX.0.Y for minor updates)

3.0 Architectural Strategies

- **Client-server web architecture** with REST APIs.
- **Modular pipeline**: ingestion → validation → normalization → storage → ML → reporting.
- **Microservice separation** for heavy ML tasks to prevent backend blocking.
- **Data-first design** with canonical schema (timeseries).
- **Security & observability**: HTTPS, encrypted storage, logging, monitoring.

4.0 System Architecture



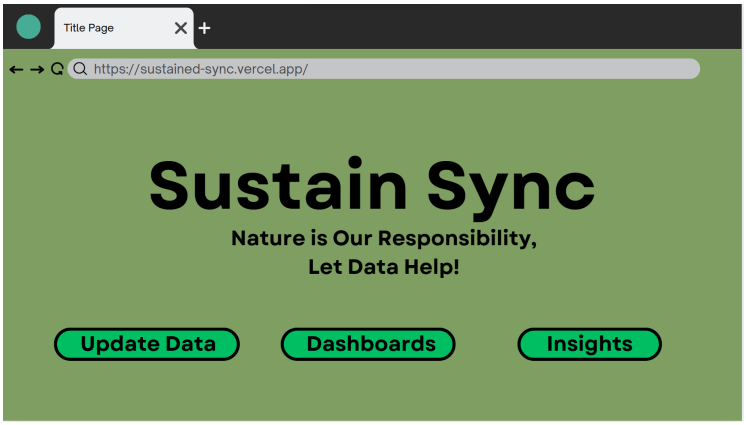
- **Frontend (React/Next.js)**: dashboards, file uploads, goal management.
- **Backend (Django REST)**: authentication, APIs, orchestration, DB ops.
- **Ingestion service**: CSV validation (pandas).
- **Database (PostgreSQL)**: normalized timeseries, metadata, object storage for raw uploads.
- **Forecasting Service**: LSTM/Prophet for timeseries prediction.
- **Co-Benefit Engine (SSCE)**: statistical correlations + LLM for narrative suggestions.

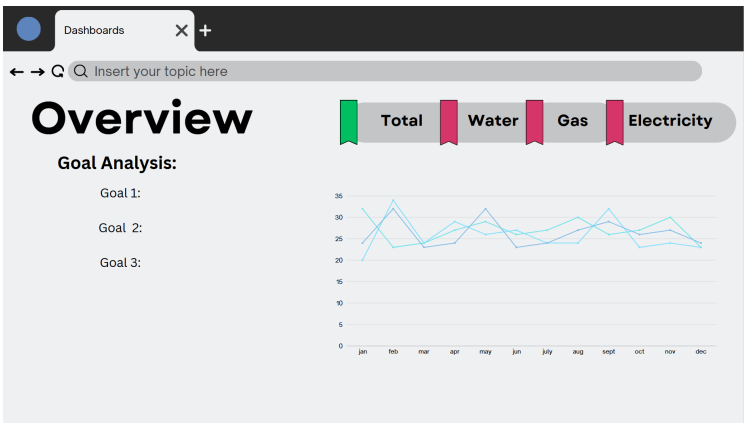
5.0 Detailed System Design

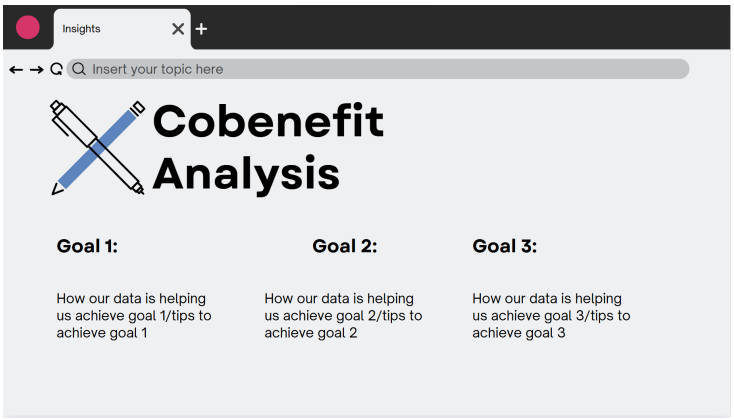
5.1 Frontend (Web Client)

- **Classification**: Subsystem (React/Next.js, Browser UI).
- **Definition**: Provides file upload, dashboard visualization, and goal management. Calls REST API endpoints exposed by backend.
- **Constraints**: Requires modern browser and responsive layout.
- **Interface/Exports**:
    - POST /api/upload (file: CSV)
    - GET /api/datasets
    - GET /api/forecasts/{id}
    - POST /api/goals
    - GET /api/SSCEsuggestions

Example UI:







5.2 Backend (Django REST API)

- **Classification**: Subsystem (Django + Django REST Framework).
- **Definition**: Auth, request routing, ingestion orchestration, DB persistence.
- **Constraints**: Enforce authentication; atomic ingestion transactions.
- **Resources**: Django and PostgreSQL.
- **Interface/Exports**:
    - POST /api/upload
    - GET /api/datasets/{id}
    - POST /api/forecast
    - GET /api/forecasts/{id}
    - GET /api/SSCEsuggestions

5.3 Ingestion Service

- **Classification**: Module (Python parser/validator).
- **Definition**: Parses CSVs (via pandas), maps to canonical schema.
- **Constraints**:
- Accept only Sustain Sync CSV template.
- Validate datatypes (datetime, date, float, varchar, enum, string).
- Minimum of 12 months data required for forecasting
- **Resources**: pandas, numpy, regex validators.
- **Interface/Exports**: Normalized record (example):

```
{
  "bill_id": "uuid",
  "bill_type": "Power|Gas|Water",
  "upload_ts": "2025-01-15T12:00:00Z",
  "bill_date": "2024-12-01",
  "units_of_measure": "kWh",
  "consumption": 345.67,
  "service_start": "2024-12-01",
  "service_end": "2024-12-31",
  "provider": "Georgia Power",
  "city": "Atlanta",
```

```
  "state": "GA",
  "zip": "30318",
  "cost": 125.55,
}
```

5.4 Database (PostgreSQL)

- **Classification**: Subsystem (Data persistence layer).
- **Definition**: Stores ingested utility bills, forecast outputs, and user goals.
- **Constraints**:
    - All timestamps stored in UTC.
    - Primary key uniqueness (bill_id).
    - Validation constraints (non-negative consumption/cost, valid zip codes).
- **Schema:**

FIELD	DATATYPE (POSTGRES)	NOTES
BILL_ID	UUID (PK)	Unique ID
BILL_TYPE	ENUM('Power','Gas','Water')	Utility category
TIMESTAMP_UPLOAD	TIMESTAMP	Upload time (UTC)
BILL_DATE	DATE	Date on bill
UNITS_OF_MEASURE	VARCHAR	e.g., kWh, therms
CONSUMPTION	DOUBLE PRECISION	Must be ≥ 0
SERVICE_START	DATE	Optional
SERVICE_END	DATE	Must ≥ service_start
PROVIDER	VARCHAR	Company name
CITY	VARCHAR	Location
STATE	VARCHAR(2)	US state code
ZIP	VARCHAR(5)	Must match regex ^\d{5}$
COST	NUMERIC(12,2)	Must be ≥ 0
FILE_SOURCE	TEXT	Original filename

5.5 Forecasting Service

- **Classification**: Subsystem (ML worker service).
- **Definition**: Applies Prophet/LSTM models on normalized data. Produces forecasts with confidence intervals.
- **Constraints**: Needs ≥12 months history; forecast horizons 1–12 months

- **Resources**: scikit-learn, Prophet, PyTorch.
- **Interface/Exports**:
    - Request: {metric:"Power", horizon:6}
    - Response: {forecast_id, timestamps[], yhat[], yhat_lower[], yhat_upper[], model_version}

5.6 Co-Benefit Engine (SSCE)

- **Classification**: Subsystem (Analytical + LLM integration).
- **Definition**: Computes correlations between domains (CO_2, water, biodiversity). Generates grounded narrative suggestions.
- **Constraints**: Must cite numeric evidence; avoid unsupported LLM hallucinations.
- **Resources**: numpy, Ollama/DeepSeek API.
- **Interface/Exports**:
    - Request: {goal_id}
    - Response: {goal_id, correlations:[{pair, r, p}], recommendations:[{text, confidence}]}

Development Overview

1.0 Backend Development

We used Docker to isolate dependencies and ensure consistent behavior across all environments. Backend development began by defining the ingestion pipeline and database schema, since reliable data normalization was foundational to forecasting and RAG analysis. After stabilizing CSV validation, we implemented API endpoints incrementally, validating each through manual testing and frontend integrations. We adopted a modular architecture so ingestion, forecasting, and RAG services could evolve independently without disrupting the Django REST layer. Throughout development, we refined logic based on test cases, CSV variations, and feedback from our dashboard requirements.

1.1 Core Dependencies

- Django==5.2.7
- psycopg2-binary==2.9.10
- django-cors-headers==4.0.0
- pandas==2.2.3
- numpy==1.26.4
- prophet==1.1.6
- torch==2.2.0
- sentence-transformers==2.2.2
- faiss-cpu==1.7.4
- transformers==4.35.2

1.2 CSV Ingestion

The CSV ingestion module was one of the earliest components developed and required several iterations. Early versions could parse files but lacked strong validation. We expanded this by adding row-level error reporting, duplicate detection, timezone normalization, and schema enforcement. These improvements ensured downstream systems—especially forecasting and RAG—received consistent, high-quality data. We repeatedly tested ingestion using malformed and edge-case CSV files to confirm resilience.

File: *backend/import_bills.py*

- Parses uploaded CSVs using *pandas*.
- Validates field types and converts *bill_date* to timezone-aware datetimes.
- Inserts or updates *Bill* records in the PostgreSQL database.
- Handles row-level validation with structured error messages for the frontend.

1.3 Aggregation & Metrics

Aggregation endpoints were built after ingestion was stable. These endpoints use Django ORM functions (such as *TruncMonth* and *Sum*) to compute monthly totals and utility-specific breakdowns. During development, we identified missing-month issues that affected chart rendering, so we updated aggregation logic to ensure chronological continuity and predictable responses for the frontend.

File: *backend/SustainSync/views.py*

- Implements monthly and metric endpoints using Django ORM aggregation (*TruncMonth + Sum*).
- Returns JSON responses with monthly totals and per-utility breakdowns.

1.4 Forecasting & RAG (Retrieval-Augmented Generation)

We developed forecasting and RAG as separate Python modules to avoid coupling model logic directly to Django views. Forecasting initially used Prophet exclusively, but testing on smaller datasets revealed accuracy concerns—leading to the implementation of a linear regression fallback. We created a preprocessing pipeline to convert raw monthly data into the format Prophet expects.

RAG development required substantial experimentation with prompt engineering, embeddings, and retrieval logic. Early outputs suffered from hallucinations, so we introduced structured prompts, strict JSON formatting, and post-processing rules. Moving the RAG workflow into a dedicated module (rag.py) improved maintainability and allowed rapid iteration without affecting API stability.

File: *backend/llm/rag.py*

- Prepares monthly time series from normalized bill data.
- Uses **Prophet** for primary forecasting of both cost (*yhat*) and usage (*yhat_usage*).
- Implements a fallback **linear regression** model if data is insufficient.
- Produces summarized AI insights for the frontend dashboard.
- Utilizes Ollama3.2 for analysis RAG model

1.5 Key Endpoints

ENDPOINT	METHOD	DESCRIPTION
/API/DASHBOARD/METRICS/	**GET**	Returns aggregate totals by bill type
/API/DASHBOARD/MONTHLY/	**GET**	Monthly time series of costs and consumption
/API/FORECAST/?PERIODS=12	**GET**	Forecast results and summaries
/API/BILLS/UPLOAD/	**POST**	Upload CSV and return results/errors
/API/BILLS/TEMPLATE/	**GET**	Download CSV template

2.0 Frontend Development

Frontend development began with custom React components, but as the dashboard grew more complex, UI consistency and maintainability became challenges. Midway through development, we refactored the entire frontend to use Material UI (MUI). This decision provided a unified design system, reduced styling overhead, and allowed us to rapidly assemble polished layouts for charts, tables, upload tools, and insights panels. Component structure was reorganized around MUI's layout primitives (Grid, Box, Card), which improved readability and responsiveness.

We followed a component-first development process:

- Build standalone chart/table components.
- Connect them to backend endpoints.
- Add dynamic state updates so CSV uploads, goals, and forecasts immediately refresh UI elements.
- Integrate RAG insights in a structured, interactive panel.

This iterative approach made frontend development more efficient and allowed quick adjustments based on user testing.

2.1 Key Components

- Charts: SimpleLineChart and MultiSeriesForecastChart visualize historical vs. forecast data using SVG and responsive design.
- Tables: MonthlyBreakdownTable and QuarterlyBreakdownTable compute monthly and quarterly aggregates.

- Upload: CSV upload control posts to /api/bills/upload/ and reloads metrics upon success.
- The frontend is connected to the Django backend using CORS and Axios. Each fetch updates a centralized state, re-rendering charts and tables dynamically.

3.0 Database Connection

SustainSync uses a PostgreSQL database running inside a Docker container. Django connects using environment variables defined in .env and passed to the container in docker-compose.yml.

Example Configuration:
POSTGRES_DB=sustainsync
POSTGRES_USER=admin
POSTGRES_PASSWORD=securepassword
POSTGRES_HOST=db
POSTGRES_PORT=5432

4.0 Project Setup Instructions

4.1 Folder Structure

```
SustainSync/
|
├── backend/      # Django app (API, models, endpoints)
├── frontend/     # React + Vite dashboard
├── llm/          # RAG + Prophet scripts
├── data/         # CSV and cached FAISS index
└── docker-compose.yml
```

4.2 Quickstart (Local Development)

Follow these steps to run SustainSync locally using Docker Compose. Note this requires docker desktop. Please follow installation guide beforehand from dockers website.

1. Clone the Repository

```
git clone https://github.com/Sustained-Sync-API/cs4850.git
cd cs4850/SustainSync
```

2. Create the `.env` File

Create a file named `.env` in the project root using the template below.
Do not commit this file — it contains sensitive information used only for local development.

Template:
 # Local .env for development. Fill in real values and DO NOT commit this file.

 # --- PostgreSQL Database Configuration ---
POSTGRES_DB=sustainsync
POSTGRES_USER= #add username
POSTGRES_PASSWORD= #create password
POSTGRES_HOST=db
POSTGRES_PORT=5432

 # --- CSV File Configuration ---
 # Path to the CSV file used by the import script
BILLS_CSV=./SustainSync/data/billdata.csv

 # --- Django Configuration ---
DJANGO_SECRET_KEY=replace-with-a-secret
DJANGO_DEBUG=True
FRONTEND_URL=http://localhost:3000

 # --- Optional Admin Account (auto-created at startup) ---
DJANGO_SUPERUSER_USERNAME= #add username for db
DJANGO_SUPERUSER_EMAIL= #add email
DJANGO_SUPERUSER_PASSWORD= #create password

3. Build and Start the Application

docker compose up –build

This will launch the following containers:

- web: Django backend
- db: PostgreSQL database
- frontend: React/Vite dashboard (optional in local builds)

4. Access the Application

SERVICE	URL
FRONTEND DASHBOARD	*http://localhost:3000*
DJANGO ADMIN PANEL	*http://localhost:8000/admin/*
API ROOT ENDPOINT	*http://localhost:8000/api/*

5. Shutdown

When finished, stop all services with:

docker compose down

Software Test Plan

1.0 Purpose

The purpose of this Software Test Plan is to define the testing scope, objectives, responsibilities, strategy, and criteria for Sustain Sync. This document outlines what will be tested, how testing will be performed, required resources, dependencies, and how testing success will be evaluated.

1.1 Scope

This plan covers functional testing for Sustain Sync's MVP, including CSV ingestion, goal management, forecasting interactions, and RAG-based recommendation updates. It defines in-scope and out-of-scope areas, test methodology, environment needs, and administrative requirements.

2.0 Testing Summary

2.1 Scope of Testing

In Scope

- CSV ingestion validation
- Goal creation and editing
- Dashboard update verification
- RAG-based analysis updates
- Basic regression on ingestion and goal workflows

Out of Scope

- PDF parsing (future work)
- Load/performance testing
- Authentication & role-based access
- Full forecasting model validation

3.0 Analysis of Scope and Test Focus Areas

3.1 Release Content

This release includes CSV ingestion, goal tracking, forecasting integration, and AI-based co-benefit analysis.

3.2 Regression Testing

Regression testing includes CSV ingestion and goal update workflows to confirm previous functionality remains stable.

3.3 Test Cases

Test Case ID	Description	Expected Result
TC-01: Invalid CSV Upload	User uploads a CSV file that does **not** match the Sustain Sync template.	Upload fails; system returns an error message indicating invalid format.
TC-02: Valid CSV Upload	User uploads a correctly formatted CSV file.	Upload succeeds; dashboard updates with newly ingested data.
TC-03: Edit Existing Goal	User edits an existing sustainability goal.	Goal updates successfully and the RAG model uses the new value in analysis.
TC-04: Add New Goal	User adds a new sustainability goal.	New goal is stored, persisted, and included in RAG analysis alongside existing goals.

4.0 Test Strategy & Procedures

4.1 Responsibility

Unit Testing: Zaid Khan
Integration Testing: Team
UAT: Developer & PM jointly

4.2 Test Types & Approach

Manual functional testing is the primary approach. Regression testing performed after changes.

4.3 Build Strategy

Single deployment via Docker Compose.

4.4 Test Execution Schedule

Testing executed from 11/16/2025-11/23/2025.

4.6 Testing Tools

Manual testing; logs via Docker; Excel for results tracking.

4.7 Procedures

Test Case 1 – Invalid CSV Upload

Objective: Ensure system rejects improperly formatted CSV files.

Steps

1. Start the application using docker compose up.
2. Navigate to http://localhost:3000.
3. Click **Upload CSV**.
4. Select an invalid CSV file (wrong headers, missing columns, incorrect date format).
5. Click **Submit**.

Expected Result

- System displays: **"Upload Failed: Invalid Format"**
- No new records appear in the dashboard.
- Backend logs show a validation error.

Test Case 2 – Valid CSV Upload

Objective: Ensure ingestion works and dashboard updates.

Steps

1. Start the application using docker compose up.
2. Navigate to http://localhost:3000.
3. Click **Upload CSV**.
4. Select the valid billdata.csv file.
5. Click **Submit**.
6. Refresh dashboard.

Expected Result

- System displays confirmation message.
- Dashboard shows updated metrics and graphs.
- PostgreSQL contains newly inserted rows.

Test Case 3 – Edit Existing Goal

Objective: Ensure goal edits update and RAG analysis persists with updates.

Steps

1. Navigate to the **Goals** page.
2. Select an existing goal.
3. Click **Edit Goal**.
4. Update the goal's numeric target or timeline.
5. Save changes.
6. Refresh dashboard.

Expected Result

- Updated goal replaces the old one.
- RAG output references only the *new* goal.
- Dashboard reflects new goal progress.

Test Case 4 – Add New Goal

Objective: Ensure new goals update and influence RAG analysis.

Steps

1. Navigate to the **Goals** page.
2. Click **Add Goal**.
3. Enter goal name, target value, and timeline.
4. Save the new goal.
5. Refresh dashboard.

Expected Result

- New goal appears in the goal list.
- RAG analysis incorporates all active goals.
- Dashboard updates progress indicators.

5. Test Environment Plan

5.1 Environment Details

Docker containers: Django, PostgreSQL, React frontend.

5.2 Assumptions & Dependencies

Assumes local Nvidia GPU available; assumes stable Docker environment.

5.3 Entry & Exit Criteria

Entry: environments running; valid test data prepared.
Exit: all high-priority tests pass; defects logged/resolved.

6. Test Data

- Valid CSV: Sustain Sync billdata.csv

- Invalid CSV: malformed CSV
- Goals: generic goal and edited version

Software Test Report (STR)

1.0 Software Test Report – Summary Table

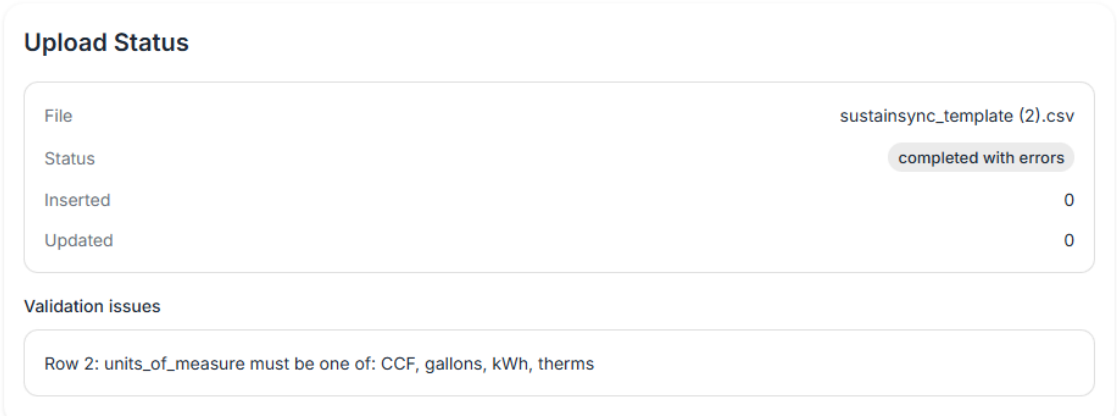
Requirement/Test	Pass	Fail	Severity
TC-01 Upload Invalid CSV	Yes		Critical
TC-02 Upload Valid CSV	Yes		Critical
TC-03 Edit Goal	Yes		Critical
TC-04 Add New Goal	Yes		Major

1.1 Detailed Results

TC-01: Invalid CSV Upload

Result: System rejected invalid CSV.
Status: Pass

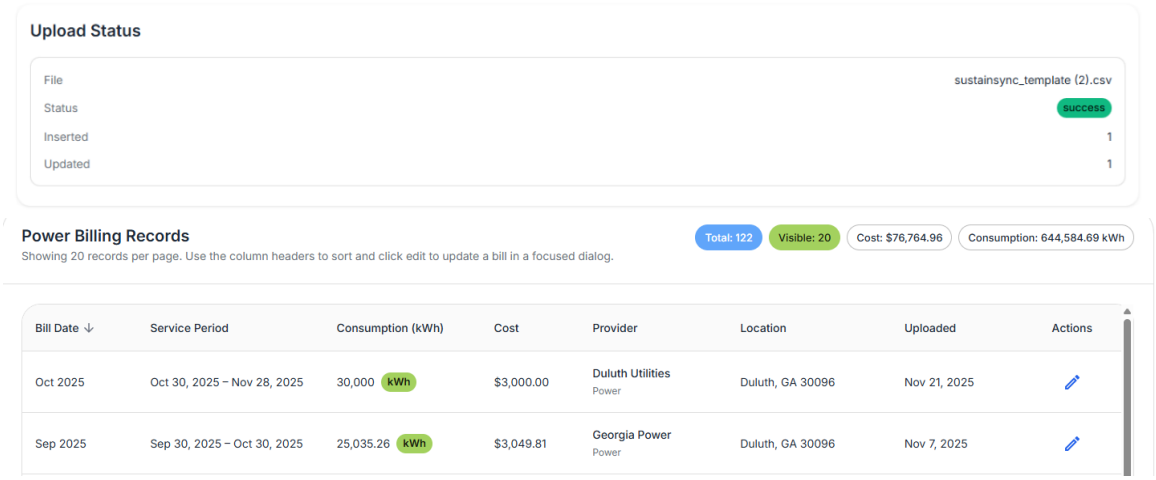
	A	B	C	D	E	F	G	H	I	J	K	L
1	bill_id	bill_type	bill_date	service_sta	service_en	units_of_measure	consumpt	cost	provider	city	state	zip
2	12345	Power	11/1/2025	10/1/2025	11/1/2025	BogusUnit	1200	450.32	Duluth Uti	Duluth	GA	30096
3												



TC-02: Valid CSV Upload

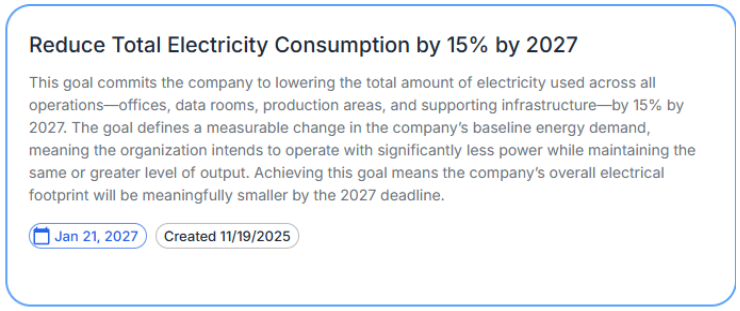
Result: System ingested CSV and updated dashboard.
Status: Pass

	bill_id	bill_type	bill_date	service_sta	service_enc	units_of_measure	consumpt	cost	provider	city	state	zip
1												
2	12345	Power	11/1/2025	10/31/2025	11/29/2025	kWh	30000	3000	Duluth Uti	Duluth	GA	30096
3												

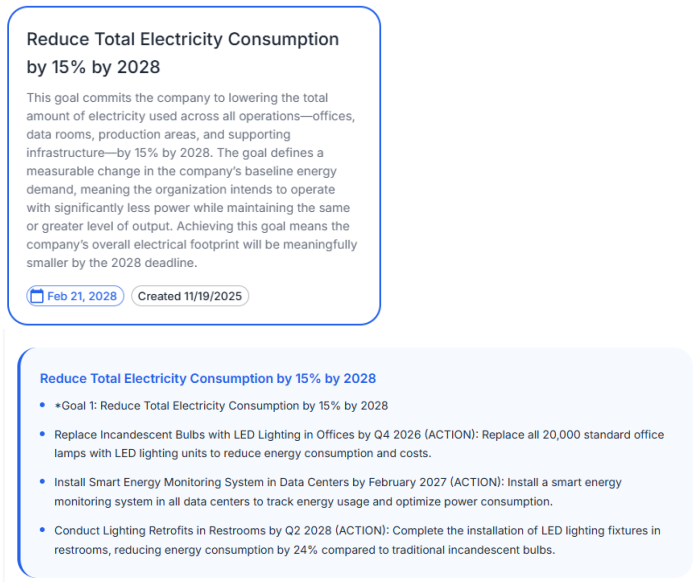


TC-03: Edit Goal

Result: Goal updated and reflected in RAG.
Status: Pass
OLD:

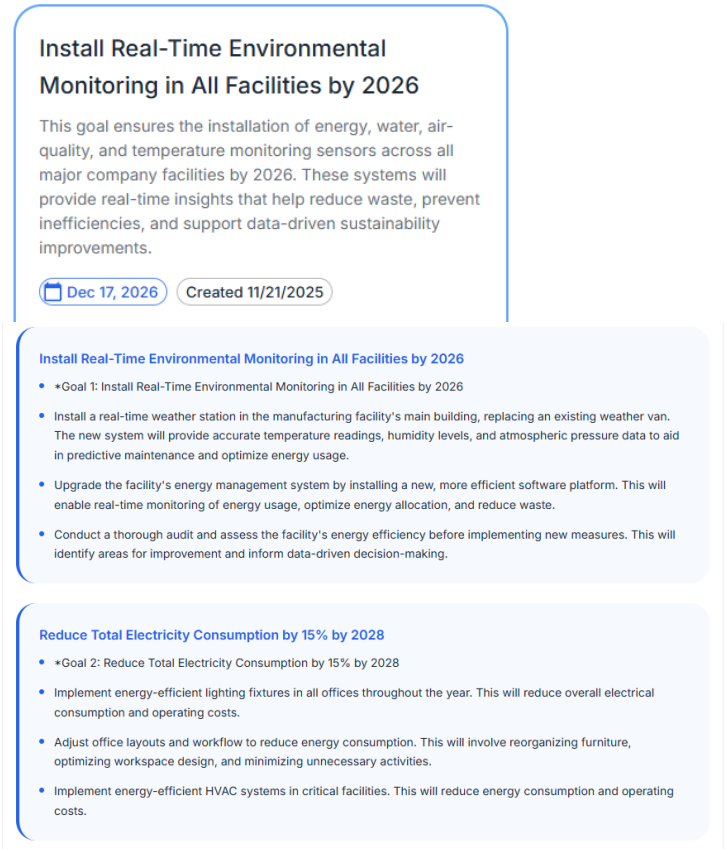


New:



TC-04: Add New Goal

Result: New goal added, RAG updated.
Status: Pass



Version Control

- **GitHub Releases:** We will use GitHub's release feature to manage and track project versions. We will start with v1.0.0 for initial pre-release without functionality.

- **Major Updates:** Major releases will follow the format v1.0.0, v2.0.0, v3.0.0, etc. These indicate significant changes such as new functionality, breaking changes, or architectural updates.
    - v1.0.0 indicates our initial run throughs for git setup, alongside general structure and attempts.
    - v2.0.0 indicates our presentation, with major RAG fixes needed.
    - v3.0.0 indicates our final MVP submission for C-Day, and the final submission.

- **Minor Updates:** Minor updates will follow the format vX.0.1, vX.0.2, vX.0.3, etc. These will capture incremental changes such as bug fixes, small enhancements, or non-breaking improvements.

Conclusion

Overall, this project reinforced the importance of establishing consistent, data-driven methods for evaluating corporate sustainability, especially given the absence of widely adopted standards across industries. By centering the MVP on universally available utility data and transforming it through a structured RAG pipeline, forecasting models, and AI-based correlation analysis, the system demonstrated a practical path forward for organizations seeking clearer insight into their environmental impact. Although the initial vision was broad, the project constraints required a focused scope that ultimately resulted in a narrow and functional solution.

Hardware limitations, most notably the need to rely on a smaller Ollama model due to an older 8 GB GPU, influenced the complexity of LLM outputs but also highlighted the trade-offs inherent in deploying AI systems on constrained infrastructure. Despite these challenges, the project delivered meaningful progress in areas such as prompt engineering, API design, data ingestion, and containerized development. The experience gained across these technical domains, paired with the structured planning required throughout the semester, contributed to a more disciplined and scalable development approach.

In its final form, the MVP successfully integrates data ingestion, forecasting, correlation analysis, and user-focused interfaces, providing a foundation for a standardized sustainability intelligence platform. The insights gained through this process position the system for future expansion as more datasets, stronger models, and greater computational resources become available.

Appendix

Appendix A – Project Plan

1.0 Team Organization

Name	Role	Email
Youssef El-Shaer	Project Manager	yewebdesign2002@gmail.com
Zaid Khan	Software Engineer	zaidkhan5213@gmail.com
Sharon Perry	Advisor	sperry46@kennesaw.edu

1.1 Roles

Youssef El-Shaer – Project Manager

- Breaks down larger parts of the project into bite-sized chunks
- Checks that each team member is submitting work on time
- Acts as link between the team and your professor
- Emails professor with team's progress and questions once per week or as needed
- Checks the almost-finished deliverable to make sure that it follows assignment criteria
- Creates an agenda (list of topics and decisions to be made) for each meeting.

Zaid Khan – Software Engineer

- Oversees the testing and bug identification for the assigned project. As well as handling the merging of project classes while ensuring project integrity and adherence to project requirements.
- Work with the project manager to designate priority to development tasks, and to lead/support development efforts.
- Work with the team to plan and implement software solutions and peer review other developers' code.

1.2 Abstract

Sustain Sync applies machine learning to analyze and forecast environmental impacts. The system ingests structured and semi-structured data from utility bills, and other sources in PDF, CSV, and Excel formats, normalizing them into a consistent schema for processing.

The MVP includes two components: (1) a forecasting module using machine learning to predict trends in CO_2 emissions, water consumption, and energy usage, and (2) a correlation module leveraging large language models to identify relationships between sustainability actions across these domains CO_2 emissions, biodiversity, and water. Contextual variables such as location and energy source are also considered.

The system provides a web interface to upload data, view historical dashboards, and an Api to output structured results, supporting integration with dashboards or reporting pipelines.

Success criteria for the MVP include ingestion of multiple datasets, generation of forecasted trends, and identification of at least one cross-domain correlation.

1.3 Website

- We will host our website on GitHub pages, https://sustainsync.github.io/SustainSync-Website.

1.4 Deliverables

- Team/Project Selection document (Individual Assignment)
- Weekly Activity Reports (WARs – Individual Assignment)
- Team Status Report (TSR – Group Assignment)
- Peer Reviews (Individual Assignment)
- Project Plan (Group Assignment)
- SRS, SDD, STP & Dev Doc (Group Assignment)
- Prototype Presentation for Peer Review (Group Assignment)
- Final Report Package (Group Assignment)
    - Final Report (Group Assignment)
    - Source Code - Sustainability Forecasting & Correlation Engine (Group Assignment)
    - Website (Group Assignment)
    - Video Demo (Group Assignment)
- C-Day Application/Submission

1.5 Group Meeting Schedule Date/Time

Team Meetings	Tuesday/Thursday 4:45-6:15pm
Stakeholder Status Meeting	Thursday 4:15-4:30pm
Weekly Check-in	Sunday 8-8:15pm

1.6 Collaboration & Communication Plan

- Discord – team meetings, stakeholder status meetings, and general conversation.
- Text/Call – for urgent communications.
- Google Workspace – will be used to host temp docs.
- Github & Github Projects – Source control & Kanban board for issue tracking.
- Microsoft Word – final docs.
- Canva – for graphic design.

1.7 Project Schedule & Task Planning

Project Name: Sustain Sync
Report Date: 8/30/25

					Milestone #1				*Milestone #2*				*Milestone #3*				C-Day	
Deliverable	**Tasks**	**Complete%**	**Current Status Memo**	**Assigned To**	**08/21**	**08/26**	**08/28**	**09/02**	**09/09**	**09/14**	**10/05**	**10/26**	**11/01**	**11/16**	**11/30**	**12/03**	**12/04**	**12/08**
Requirements	Define requirements	60%	In Progress	Zaid & Youssef	3													
	Review requirements with SH	0%		Zaid & Youssef		7	7											
	Get sign off on requirements	0%		Zaid & Youssef			4	4										
Project design	Define tech required *	0%		Zaid & Youssef					8									
	Database design	0%		Zaid & Youssef					6									
	API Design	0%		Zaid & Youssef						8								
	UI Design	0%		Zaid & Youssef						10								
	Develop working prototype	0%		Zaid & Youssef							30	30						
	Test prototype	0%		Zaid & Youssef							10	10						
Development	Review prototype design	0%		Zaid & Youssef									32	32				
	Rework requirements	0%		Zaid & Youssef									12	12				
	Document updated design	0%		Zaid & Youssef											30			
	Test product	0%		Zaid & Youssef											10	2		
Final report	Presentation preparation	0%		Zaid & Youssef												8		
	Poster preparation	0%		Zaid & Youssef												6	4	
	Final report submission to D2L	0%		Youssef														2
			Total work hours	**287**	**3**	**7**	**11**	**4**	**14**	**18**	**40**	**40**	**44**	**44**	**40**	**16**	**4**	**2**
								SRS		**SDS**		**DD**		**Present**	**Test Plan**	**Prep**	**C-Day**	**Final**

* formally define how you will develop this project including source code management

Legend
Planned
Delayed
Number Work: man hours

Appendix B – Glossary & Acronyms

- **API (Application Programming Interface):** A defined method for communication between software components.
- **SSCE (Sustain Sync Co-Benefit Engine):** The system's AI-driven module for analyzing sustainability correlations and generating recommendations.
- **LLM (Large Language Model):** A machine-learning model used to interpret data and generate text-based insights.
- **CSV Template:** A predefined data format used to standardize utility bill ingestion within Sustain Sync.
- **BYOD (Bring Your Own Data):** The requirement that users upload their own historical utility data for forecasting and analysis.

Appendix C – Bibliography

Technical References

- GeeksforGeeks. (n.d.). *Strategies for Schema Design in DBMS*. Retrieved from https://www.geeksforgeeks.org/dbms/strategies-for-schema-design-in-dbms/
- GeeksforGeeks. (n.d.). *Django Tutorial*. Retrieved from https://www.geeksforgeeks.org/python/django-tutorial/
- Mathur, D. (2024). *Django and Microservices Architecture: A Comprehensive Guide (Part 1/7)*. *Medium*. Retrieved from https://medium.com/@mathur.danduprolu/django-and-microservices-architecture-a-comprehensive-guide-part-1-7-6505e42cc38d

Domain References

- Georgia Power. (n.d.). *Interactive Sample Bill*. Retrieved from https://www.georgiapower.com/residential/billing-and-rate-plans/billing-options/understanding-your-bill/interactive-sample-bill.html
- Cobb County Water System. (n.d.). *Sample Water Bill*. Retrieved from https://www.cobbcounty.gov/water/customer-service/water-bill-sample
- Georgia Natural Gas (GNG). (n.d.). *Understand My Bill*. Retrieved from https://gng.com/my-gng/understand-my-bill